

Modernes C++ — Schulung

Schwerpunkte: OOP, Templates, Smart Pointer,
Modern C++, Multithreading, Unit Testing

Dr.-Ing. Grigory Devadze

2026-04-27

Agenda

1. Kurzer syntaktischer Überblick (Grundlagen)
2. STL — Container, Algorithmen, Iteratoren *(+ Übung)*
3. **Funktionen und Parameterübergabe**
4. **Vererbung & Polymorphismus**
5. **Dynamische Speicherverwaltung**
6. **Smart Pointer**
7. **Generische Programmierung (Templates)**
8. **Exception Handling**
9. **C++11/14/17/20 Features**
10. **Multithreading** *(inkl. Qt-Threading)*
11. **Unit Testing**
12. *Optional:* Code-Analyse, Profiling, Coding-Standards

Teil 0 — Warum C++?

Ein kurzer Blick auf Geschichte, Einsatzgebiete und Motivation.

Eine kurze Geschichte

Jahr	Meilenstein
1969	Ken Thompson & Dennis Ritchie (Bell Labs) entwickeln UNIX
1972	Ritchie erfindet C — typisierte Nachfolgersprache zu B
1979	Bjarne Stroustrup beginnt mit <i>"C with Classes"</i> (Bell Labs)
1985	Erste kommerzielle Veröffentlichung als C++ (++ ist der C-Operator)
1998	C++98 — erster ISO-Standard
2003	C++03 — kleine Korrekturen
2011	C++11 — die "moderne" Revolution: auto , Lambdas, Move-Semantik, Smart Pointer, Threading
2014	C++14 — Feinschliff
2017	C++17 — optional , variant , string_view , Structured Bindings, parallele Algorithmen
2020	C++20 — Concepts, Ranges, Modules, Coroutines, ↔
2023	C++23 — std::print , expected , Ranges-Erweiterungen
2026	C++26 (kommend) — Reflection, std::execution , Pattern Matching

Seit C++11 erscheint **alle drei Jahre** ein neuer Standard.

Wo läuft C++? — Anwendungsgebiete

Bereich	Beispiele
Betriebssysteme	Windows, macOS, Linux-Kernel-Tools, Android-Runtime
Browser	Chrome (Blink), Firefox (Gecko), Safari (WebKit)
Compiler / Toolchains	GCC, LLVM/Clang, Swift Compiler, MSVC
Spiele-Engines	Unreal, Unity-Core, CryEngine, Frostbite
Embedded / Automotive	AUTOSAR, Steuergeräte, IoT, CAN-Stacks
Finance / Trading	Niedrig-Latenz-Systeme bei Goldman, Citadel, Jane Street
High-Performance Computing	Klimamodelle, Physik-Simulationen, CERN/LHC
AI-Frameworks (Backend)	TensorFlow, PyTorch, ONNX-Runtime
Datenbanken	MySQL, MongoDB, ScyllaDB, ClickHouse
Wissenschaft / Raumfahrt	NASA Mars Perseverance Helicopter, James Webb Telescope

*Schätzung: weltweit existieren **> 200 Mrd. Zeilen** C/C++-Code.*

Warum ist C++ relevant?

- **Performance** — direkt am Hardware, ohne JIT/GC-Overhead. C++ ist der Maßstab, an dem andere Sprachen sich messen.
- **Multi-Paradigma** — prozedural, objektorientiert, generisch, funktional in *einer* Sprache.
- **Zero-Cost-Abstraktionen** — was du nicht benutzt, kostet nichts; was du benutzt, könntest du nicht effizienter selbst schreiben.
- **Ubiquität** — von 8-Bit-Microcontrollern bis Supercomputern. Eine Sprache, alle Skalen.
- **Reife** — 40 Jahre Erfahrung, Standard-Library, Tooling (Sanitizer, Profiler, Static Analysis).
- **Aktive Evolution** — alle drei Jahre neuer Standard. C++26 mit Reflection und Pattern Matching steht vor der Tür.

Modernes vs. klassisches C++

“Modernes C++” (ab C++11) sieht **anders aus** als das C++ aus den 1990ern.

Klassisches C++ (vor C++11)

```
1 char* name = new char[64];
2 strcpy(name, "Sensor-1");
3 // ... viel Code ...
4 delete[] name;           // leicht zu vergessen
```

- Manuelle Speicher-Verwaltung
- Roh-Pointer überall
- C-Strings (`char*`)
- Verbose Schleifen mit Index

Modernes C++ (ab C++11/17/20)

```
1 std::string name{"Sensor-1"};
2 // keine Freigabe nötig – RAII
```

- RAII — Ressourcen über Destruktoren
- Smart Pointer als Default
- `std::string`, Range-based for

Teil I — Syntaktischer Überblick

Refresher: Variablen, Kontrollstrukturen, Klassen, STL.

Variablen & Typen — Kurzreferenz

```
1 // Fundamentale Typen
2 int      i = 42;
3 double   d = 3.14;
4 char     c = 'x';
5 bool     b = true;
6
7 // Feste Breite (<cstdint>)
8 std::int32_t n32 = 100'000;
9 std::uint64_t big = 1ULL << 40;
```

```
1 // Moderne Initialisierung (C++11)
2 int    x{7};                // Brace-Init, verbietet narrowing
3 auto   y = 2.5;             // Typ-Deduktion: double
4 auto   z = std::string{"hi"};
5
6 // const / constexpr
7 const   int    N    = 10;
8 constexpr int    MAX = 2 * N;    // compile-time
```

`nullptr` statt `NULL` / `0`

Klassisches `NULL` ist ein Makro für `0` — also ein `int`. Bei Funktions-Überladung führt das zu Mehrdeutigkeiten und stillem Falsch-Aufruf.

```
1 void f(int);
2 void f(char*);
3
4 f(NULL);      // ? ruft f(int) auf – wahrscheinlich nicht gewollt
5 f(0);         // ? dito
6 f(nullptr);  // ✓ ruft f(char*) auf – typsicher
```

- `nullptr` (C++11) hat einen eigenen Typ: `std::nullptr_t`
- Implizit konvertierbar zu **jedem** Pointer-Typ — aber **nicht** zu `int`
- Faustregel: in modernem C++ niemals `NULL` oder `0` für Zeiger schreiben
- Auch konvertierbar zu `bool` → `if (ptr)` funktioniert weiterhin

enum class — typsichere Aufzählungen

Klassisches `enum` ist eigentlich nur `int` mit Namen — globaler Scope, implizite Casts, Namenskollisionen. `enum class` (C++11) löst alles auf einmal.

```
1 enum      Color { Red, Green, Blue };           // alt: Red, Green, Blue global
2 enum class Mode { On, Off, Standby };           // neu: Mode::On, gescopet
3 int n = Red;                                     // alt: implizit OK ⚠
4 int m = Mode::On;                                // neu: FEHLER — kein impliziter Cast
5 int k = static_cast<int>(Mode::On);             // neu: Cast nur explizit
6 enum class Status : std::uint8_t { Ok, Warn }; // eigener Underlying-Typ
```

- **Gescopet + typsicher** — keine Namenskollisionen, keine impliziten Casts nach/von `int`
- **Vorwärtsdeklarierbar** wenn Underlying-Typ angegeben ist (Header-Kompilier-Vorteile)
- **Faustregel:** in neuem Code immer `enum class`

auto & decltype — Typ-Deduktion

*Lass den Compiler den Typ bestimmen, **wenn er aus dem Kontext eindeutig ablesbar ist**. Spart Tipparbeit, vermeidet implizite Konvertierungen.*

```
1 // auto – Typ wird aus dem Initializer abgeleitet
2 auto i = 42;                // int
3 auto d = 3.14;              // double
4 auto s = std::string{"hi"};  // std::string (NICHT: const char*)
5 auto& ref = container[0];    // Referenz
6 const auto& cref = container[0]; // const-Referenz, keine Kopie
7 // decltype – übernimmt den Typ eines Ausdrucks (auch ohne Auswertung)
8 int x = 0;
9 decltype(x) y = 5;          // int
10 decltype(x + 1.0) z = 1;    // double (Ausdruck wird nicht ausgeführt)
11 // Trailing return type – Rückgabetyt hängt vom Parameter ab
12 template <typename T, typename U>
13 auto add(T a, U b) → decltype(a + b) { return a + b; }
```

- Wann **auto** ? Iterortypen, Lambda>Returns, lange Template-Typen
- Wann **explicit**? Public-API / Signaturen, wenn der Typ als Doku dient
- **auto** deduziert ohne **&** / **const** → bei Range-for fast immer **const auto&**

Uniform Initialization mit `{}`

Eine Syntax für **alle** Initialisierungen: fundamentale Typen, Klassen, Container, Aggregates.
Verbietet zusätzlich **stille Genauigkeitsverluste**.

```
1 int          x{7};                // Skalar
2 std::vector<int> v{1, 2, 3, 4};    // Container – Initializer-List
3 std::map<std::string,int> m{ {"a",1}, {"b",2} }; // verschachtelt
4 struct Point { double x, y; };
5 Point p{1.0, 2.0};                // Aggregate
6 Sensor s{"temp-1", 30.0};          // Konstruktor
7
8 // Narrowing – nur Brace-Init schlägt Alarm
9 int n{3.14};                      // FEHLER: double → int
10 int o = 3.14;                    // stillschweigend (= 3)
11
12 // Vector-Falle: () vs {}
13 std::vector<int> a(10, 5);         // 10 Elemente, alle 5
14 std::vector<int> b{10, 5};        // 2 Elemente: 10 und 5
```

- **Faustregel:** in neuem Code überall `{}` — außer wenn ein Konstruktor mit Größen-Argumenten gemeint ist (`vector(size, value)`)
- Verhindert die *Most Vexing Parse* (`Widget w();` ist eine Funktions-**Deklaration!**)

Kontrollstrukturen — Kurzreferenz

```
1 // if mit Init-Statement (C++17)
2 if (auto it = map.find(k); it != map.end()) {
3     std::cout << it->second << '\n';
4 }
5
6 // switch
7 switch (mode) {
8     case Mode::A: /* ... */ break;
9     case Mode::B: [[fallthrough]];
10    case Mode::C: /* ... */ break;
11    default:      /* ... */;
12 }
13
14 // Schleifen
15 for (int i = 0; i < n; ++i)          { /* klassisch */ }
16 for (const auto& v : container)      { /* range-based */ }
17 while (!done) { /* ... */ }
```

Range-based for : Lesender Zugriff auf jedes Element, ohne unnötige Kopien (`const auto&` bindet als const-Referenz).

[[fallthrough]] — Beispiel

```
1 enum class LogLevel { Error, Warning, Info, Debug };
2
3 void log(LogLevel level, const std::string& msg) {
4     switch (level) {
5         case LogLevel::Debug:
6             std::cout << "[debug] ";
7             [[fallthrough]]; // → Info
8         case LogLevel::Info:   std::cout << "[info] "   << msg << '\n'; break;
9         case LogLevel::Warning: std::cout << "[warning] " << msg << '\n'; break;
10        case LogLevel::Error:  std::cout << "[error] "  << msg << '\n'; break;
11    }
12 }
```

`log(LogLevel::Debug, "x")` → `[debug] [info] x` — ohne `[[fallthrough]]` würde der Compiler warnen.

Klassen — Initialisierung

Besser: Member direkt initialisieren — statt erst default-erzeugen und dann im Konstruktor-Body zuweisen.

Gut — Member-Initializer-List

```
1 class Sensor {
2 public:
3     Sensor(std::string name, double threshold)
4         : name_{std::move(name)},
5           threshold_{threshold} {}
6 private:
7     std::string name_;
8     double threshold_;
9 };
```

Member werden direkt mit den richtigen Werten erzeugt.

Schlecht — Zuweisung im Body

```
1 class Sensor {
2 public:
3     Sensor(std::string name, double threshold) {
4         name_ = std::move(name);
5         threshold_ = threshold;
6     }
7 private:
8     std::string name_;
9     double threshold_;
10 };
```

Erst default-konstruiert, dann nochmal zugewiesen — bei `string`, `vector`, ... unnötig teuer; bei `const` /Referenz-Membere n unmöglich.

Klassen — Kapselung & const-correctness

```
1 class Sensor {
2 public:
3     Sensor(std::string name, double threshold)
4         : name_{std::move(name)}, threshold_{threshold} {}
5
6     const std::string& name() const noexcept { return name_; }
7
8     bool triggers(double value) const { return value > threshold_; }
9
10 private:
11     std::string name_;
12     double threshold_;
13 };
```

- `const std::string&` als Rückgabe — Zugriff ohne Kopie, Aufrufer darf nicht ändern
- `name() const` — Methode verändert das Objekt nicht (nur lesender Zugriff)
- `noexcept` — verspricht: keine Exception
- `bool triggers(double v) const` — `s.triggers(35.0)` → `true`, falls `v > threshold_`
- Trailing underscore (`name_`) als Konvention für Member

Legacy-C++ — was ihr im Code noch seht

Alt (C / pre-C++11)	Modern (C++11/17/20)
<code>NULL</code> , <code>0</code> als Pointer	<code>nullptr</code>
<code>enum { ... }</code>	<code>enum class { ... }</code>
<code>typedef int Foo;</code>	<code>using Foo = int;</code>
<code>new</code> / <code>delete</code> von Hand	<code>std::make_unique</code> / <code>std::make_shared</code>
Roh-Pointer als Eigentümer	<code>unique_ptr<T></code> , <code>shared_ptr<T></code>
<code>printf("%d", x)</code>	<code>std::format("{} ", x)</code> / <code>std::print</code> (C++23)
C-String <code>char*</code> , <code>strcmp</code>	<code>std::string</code> , <code>std::string_view</code>
C-Array <code>T arr[N]</code>	<code>std::array<T, N></code> , <code>std::vector<T></code>
<code>throw()</code> / dyn. Exception-Spec	<code>noexcept</code>
<code>register</code> , Trigraphs, <code>auto_ptr</code>	entfernt (C++17)
Funktions-Macros (<code>#define MAX(...)</code>)	<code>constexpr</code> -Funktion / Template

Faustregel: linke Spalte → rechte Spalte ist fast immer der bessere Refactor — aber **erst nach Tests** und nur wenn es im Scope ist.

Teil II — Standard Template Library (STL)

Container, Algorithmen, Iteratoren — und eine Übung.

Container-Übersicht

Kategorie	Container	Typischer Einsatz
Sequenz	<code>vector</code> , <code>array</code> , <code>deque</code> , <code>list</code>	Datenreihen, Puffer
Assoziativ	<code>map</code> , <code>set</code> (+ <code>multi...</code>)	sortierter Schlüssel-Zugriff, $\text{Log}(n)$
Hash-basiert	<code>unordered_map</code> , <code>unordered_set</code>	schneller Lookup, $O(1)$ amortisiert
Container-Adapt.	<code>stack</code> , <code>queue</code> , <code>priority_queue</code>	Algorithmische Datenstrukturen

Faustregel: *Standardmäßig* `std::vector` — cache-freundlich, kontiguös, schnell.

Algorithmen & Iteratoren

```
1 #include <algorithm>
2 #include <numeric>
3 #include <vector>
4
5 std::vector<int> v{5, 2, 8, 1, 9, 3};
6
7 std::sort(v.begin(), v.end());           // sortieren
8 auto it = std::find(v.begin(), v.end(), 8); // suchen
9 int sum = std::accumulate(v.begin(), v.end(), 0); // Summe
10 auto cnt = std::count_if(v.begin(), v.end(),
11                          [](int x){ return x > 3; });
12
13 std::transform(v.begin(), v.end(), v.begin(), // in-place
14               [](int x){ return x * x; });
```

Algorithmen + Iteratoren: STL-Algorithmen arbeiten nicht direkt auf Containern, sondern auf Iterator-Bereichen `[begin, end)`. `begin()` zeigt aufs erste Element, `end()` auf die Position **hinter** dem letzten.

`std::sort(v.begin(), v.end());` heißt also: *sortiere alle Elemente von* `v`.

Iterator-Kategorien

Kategorie	Kann	Typische Container
<code>input</code>	einmal vorwärts lesen	Streams
<code>output</code>	einmal vorwärts schreiben	Streams, <code>back_inserter</code>
<code>forward</code>	mehrfach vorwärts laufen	<code>std::forward_list</code>
<code>bidirectional</code>	vorwärts und rückwärts	<code>std::list</code> , <code>std::map</code>
<code>random-access</code>	direkter Zugriff wie bei Arrays	<code>std::vector</code> , <code>std::array</code>

Iteratoren verbinden Container und Algorithmen. Nicht jeder Algorithmus funktioniert mit jedem Container — z. B. braucht `std::sort` Random-Access, läuft auf `std::vector`, aber **nicht** direkt auf `std::list` (dort: `list::sort()` als Member).

Übung — STL in der Praxis

Ziel: Temperatursensor-Datenreihe verarbeiten.

1. Gegeben: `std::vector<double> temps` mit Messwerten.

2. Aufgaben:

- **Statistik:** Min / Max / Mittelwert / Median
- **Filter:** nur Werte über Schwellwert (`std::copy_if`)
- **Binning:** Histogramm 10er-Buckets (`std::map<int, int>`)
- **Top-3 Spitzenwerte** (`std::partial_sort` oder `nth_element`)

3. *Stretch:* Ausreißer via Z-Score entfernen (`std::transform` + `std::remove_if`)

Übung — Lösungsskizze

```
1 auto [min_it, max_it] = std::minmax_element(temps.begin(), temps.end());
2 double mean = std::accumulate(temps.begin(), temps.end(), 0.0) / temps.size();
3
4 std::vector<double> hot;
5 std::copy_if(temps.begin(), temps.end(), std::back_inserter(hot),
6             [t = 30.0](double x){ return x > t; });
7
8 std::map<int, int> histogram;
9 for (double x : temps) ++histogram[static_cast<int>(x) / 10 * 10];
10
11 std::vector<double> top3(3);
12 std::partial_sort_copy(temps.begin(), temps.end(),
13                        top3.begin(), top3.end(), std::greater<>{});
```


Teil III — Schwerpunkt: Funktionen & Parameterübergabe

Parameterübergabe — Faustregel

Mechanismus	Syntax	Bedeutung	Wann verwenden?
By value	<code>void f(T x)</code>	Funktion bekommt eine Kopie	Kleine Typen oder eigene Kopie nötig
By const-ref	<code>void f(const T& x)</code>	Keine Kopie, nur lesen	Große Objekte, die nicht verändert werden
By ref	<code>void f(T& x)</code>	Keine Kopie, darf verändert werden	Wenn die Funktion das Original ändern soll
By pointer	<code>void f(T* x)</code>	Adresse, kann <code>nullptr</code> sein	Optionalen Parameter oder C-APIs
By rvalue-ref	<code>void f(T&& x)</code>	Darf aus dem Argument „moven“	Move-Semantik / effiziente Ressourcenübernahme

Faustregel: Kleine Werte per Kopie, große nur-lesende Objekte per `const&`, veränderbare per `&`, optionale per Pointer, Ressourcen per Move.

Parameterübergabe — Mini-Beispiele

```
1 void print(int x);           // by value: int ist klein
2 void printName(const std::string& s); // const-ref: keine Kopie, nur lesen
3 void normalize(std::vector<double>& v); // ref: verändert das Original
4 void setSensor(Sensor* s);    // pointer: darf nullptr sein
5 void store(std::string&& name); // rvalue-ref: darf moven
```

T&& bedeutet **nicht automatisch „Ownership“**, sondern: *Die Funktion darf den Inhalt übernehmen bzw. daraus `std::move` verwenden.*

Für **Konstruktoren** ist oft die einfachere Variante praktischer — *by value + move*:

```
1 class Sensor {
2 public:
3     Sensor(std::string name) : name_{std::move(name)} {}
4 private:
5     std::string name_;
6 };
```


Funktionen — moderne Features

```
1 // Standardargument: port ist optional
2 void connect(std::string_view host, int port = 8080);
3
4 // Überladung: gleicher Name, unterschiedliche Parametertypen
5 int max(int a, int b);
6 double max(double a, double b);
7
8 // Lambda: anonyme Funktion
9 auto is_even = [](int x) { return x % 2 == 0; };
10
11 // Generic Lambda (C++14): Parametertypen werden automatisch erkannt
12 auto plus = [](auto a, auto b) { return a + b; };
13
14 // constexpr: kann zur Compile-Zeit ausgewertet werden
15 constexpr int square(int x) { return x * x; }
16 static_assert(square(5) == 25); // wirklich beim Kompilieren geprüft
```

- **Default Arguments** — Parameter optional machen
- **Überladung** — mehrere Funktionen mit gleichem Namen, unterscheidbar an den Parametertypen
- **Lambdas** — kleine Funktionen direkt an der Verwendungsstelle definieren
- **constexpr** — *kann* zur Compile-Zeit ausgewertet werden, wenn die Argumente konstant sind (nicht: *muss immer*)

Return Value Optimization — Return by Value ist meist richtig

```
1 std::vector<int> make_data() {  
2     std::vector<int> v;  
3     v.reserve(1'000'000);  
4     // ... fill ...  
5     return v;                // keine teure Kopie: NRVO oder Move  
6 }  
7  
8 auto data = make_data();    // Ergebnis wird direkt übernommen
```

Große Objekte können sicher per Wert zurückgegeben werden. Der Compiler vermeidet Kopien durch Copy Elision / NRVO oder nutzt einen günstigen Move.

- **Copy Elision** seit C++17 bei Prvalues **garantiert** — kein temporäres Objekt
- Bei lokalen Variablen wie `return v;` greift meist **NRVO**; sonst wird gemoved
- Daher: kein out-Pointer nötig, einfach **Return by value**

NRVO — Named Return Value Optimization

Der Compiler kann eine **benannte lokale Variable** direkt im Speicher des Rückgabewerts erzeugen. Dadurch entfällt die Kopie beim `return`.

```
1 std::vector<int> make_data() {  
2     std::vector<int> v;          // v ist eine benannte lokale Variable  
3     v.push_back(1);  
4     v.push_back(2);  
5     return v;                   // NRVO möglich  
6 }  
7  
8 auto data = make_data();        // data wird direkt als Rückgabewert aufgebaut
```

Faustregel beim Return lokaler Variablen:

```
1 return v;                       // gut – NRVO möglich  
2 return std::move(v);            // meist schlechter – kann NRVO verhindern
```

`std::move` beim Zurückgeben lokaler Variablen ist meist unnötig und blockiert die Optimierung.

